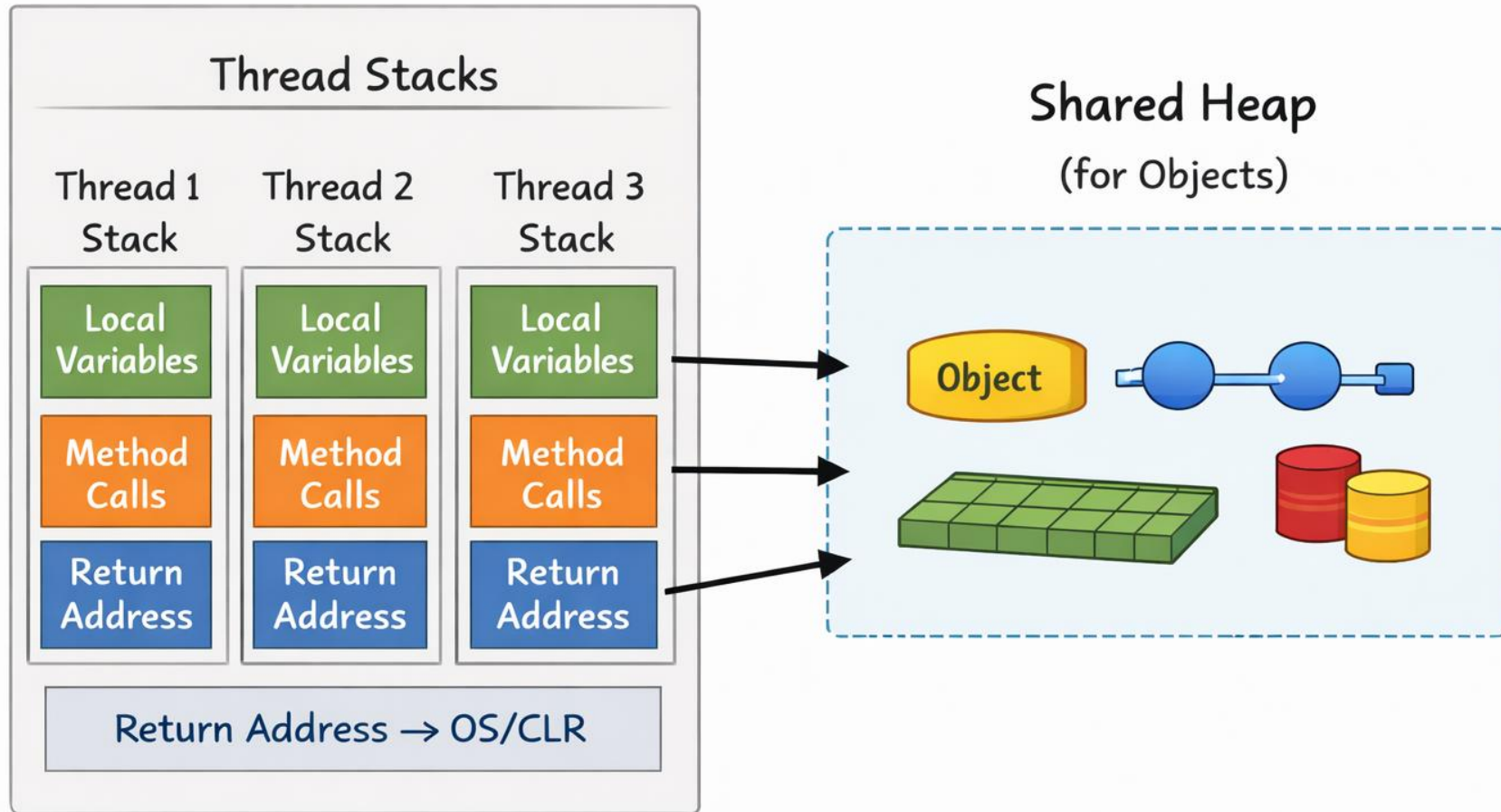


Стек на нишка

(Thread Stack)

Process

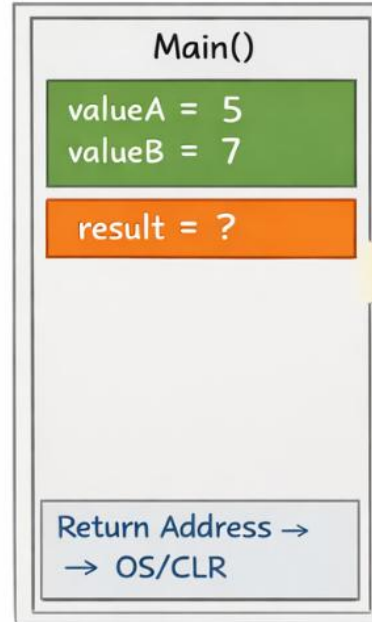


internal class Program

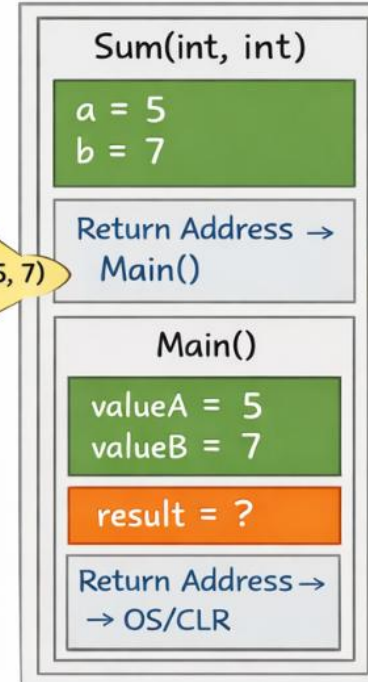
```
{  
    static int Sum(int a, int b)  
    {  
        return a + b;  
    }  
    static void Main(string[] args)  
    {  
        int valueA = 5;  
        int valueB = 7;  
  
        int result = Sum(valueA, valueB);  
  
        Console.WriteLine(result);  
    }  
}
```

Main Thread Call Stack

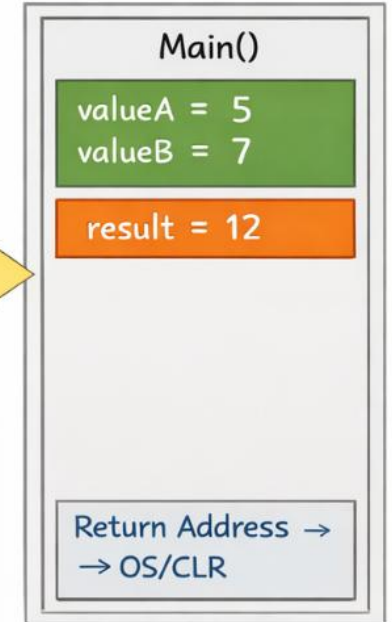
Before Calling Sum(int, int)



In Sum(int, int)



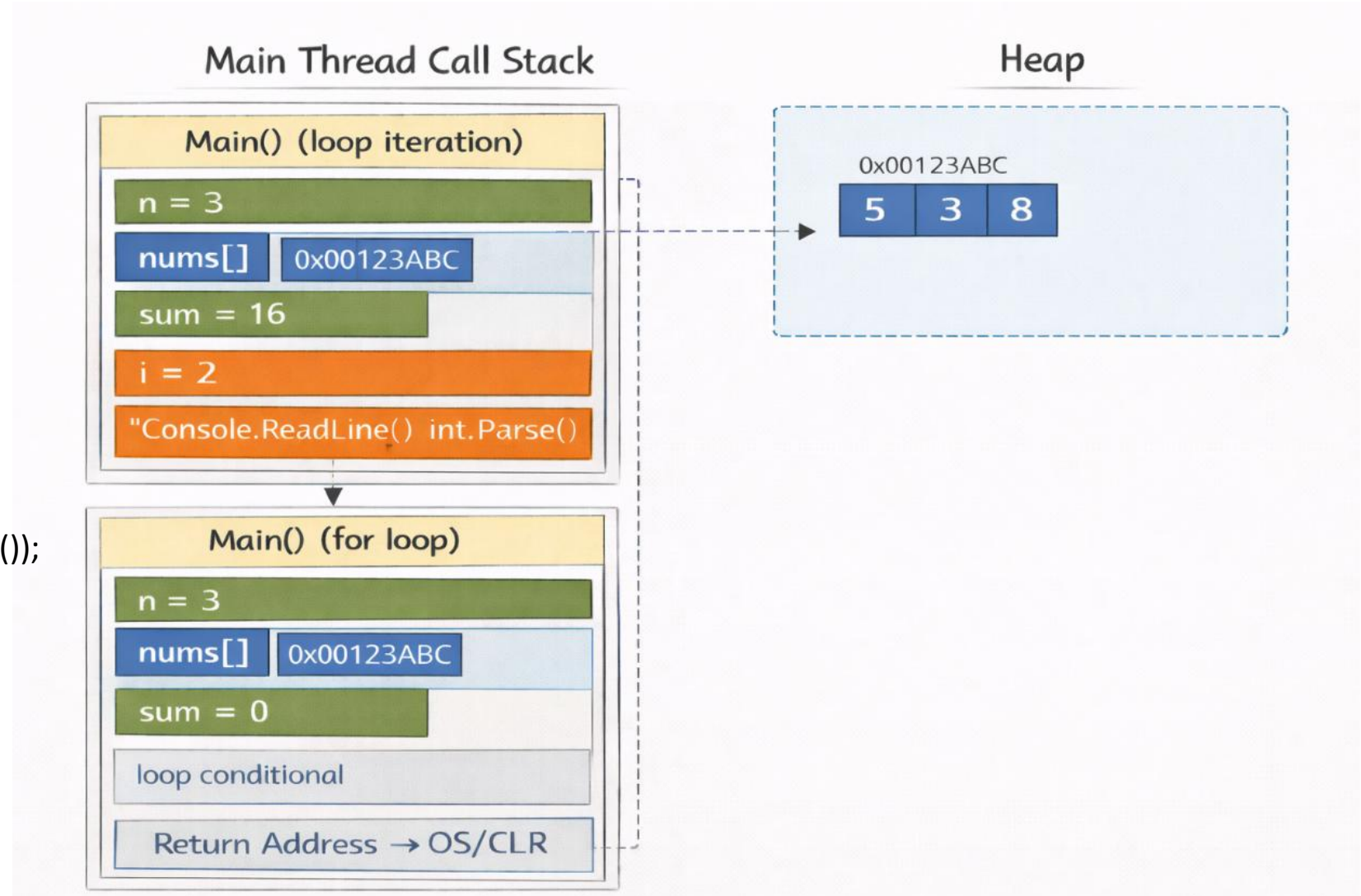
After Returning to Main()



```
static void Main(string[] args)
{
    int n = int.Parse(Console.ReadLine());
    int[] nums = new int[n];
    int sum = 0;

    for (int i = 0; i < nums.Length; i++)
    {
        nums[i] = int.Parse(Console.ReadLine());
        sum += nums[i];
    }

    Console.WriteLine(sum);
}
```



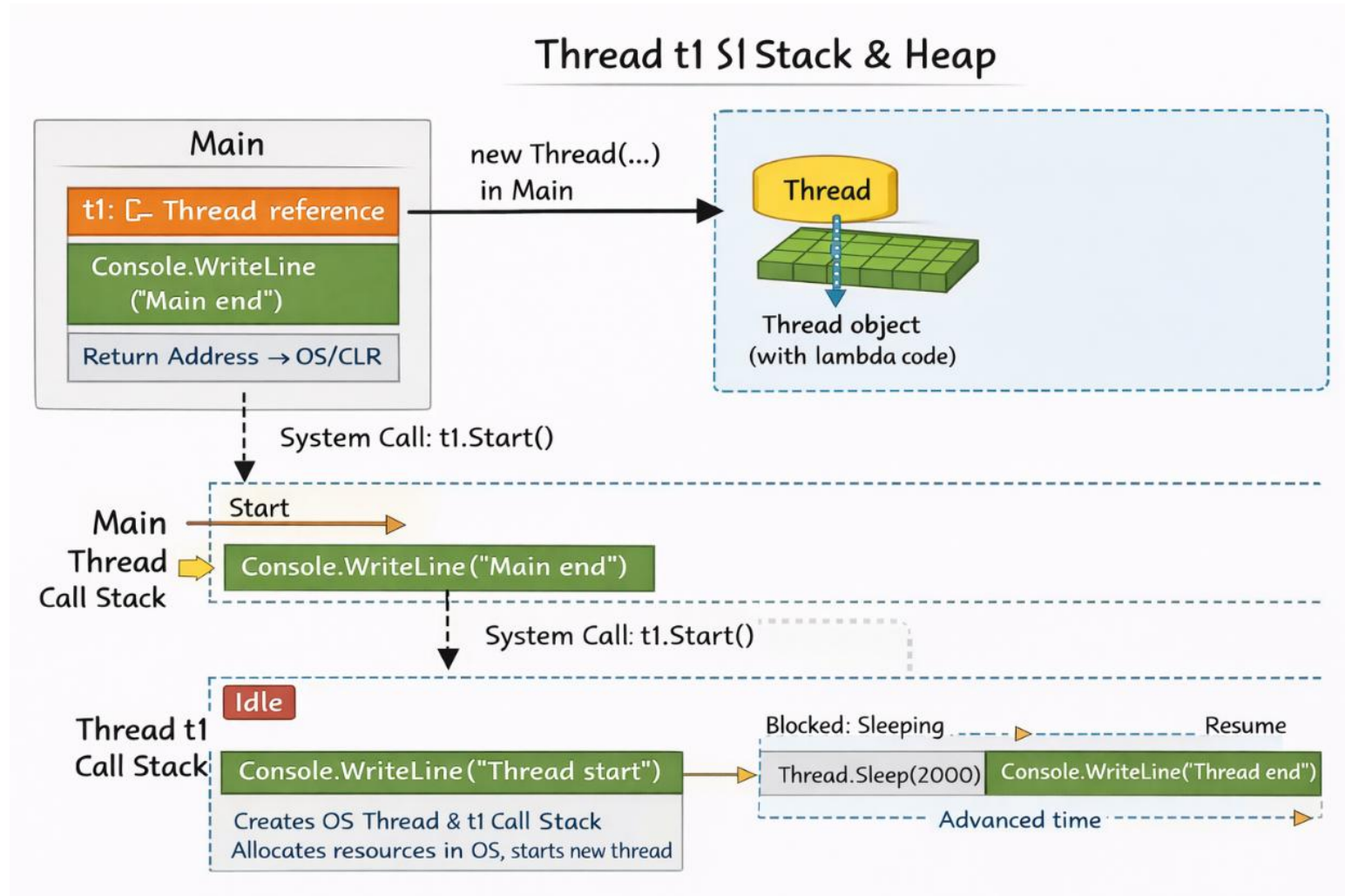
```

Thread t1 = new Thread(() =>
{
    Console.WriteLine("Thread start");
    Thread.Sleep(2000);
    Console.WriteLine("Thread end");
});

t1.Start();

Console.WriteLine("Main end");

```



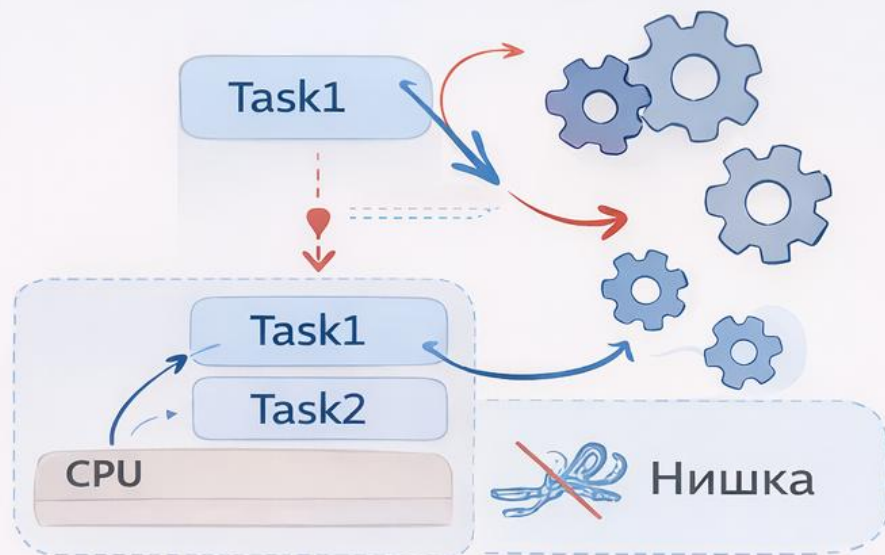
Task / async / await стартира нова нишка?

Да

Зависи от ситуацията.

Не

ThreadPool

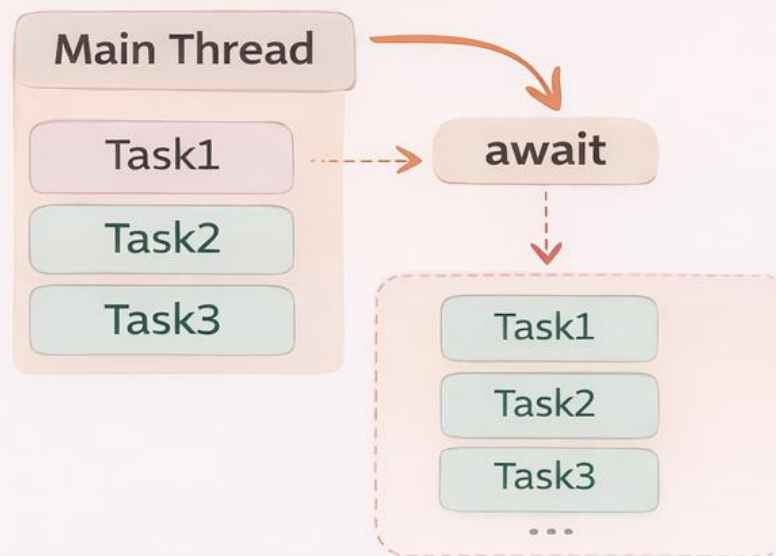


Когато задачи се изпълняват паралелно



Нишка

Main Thread

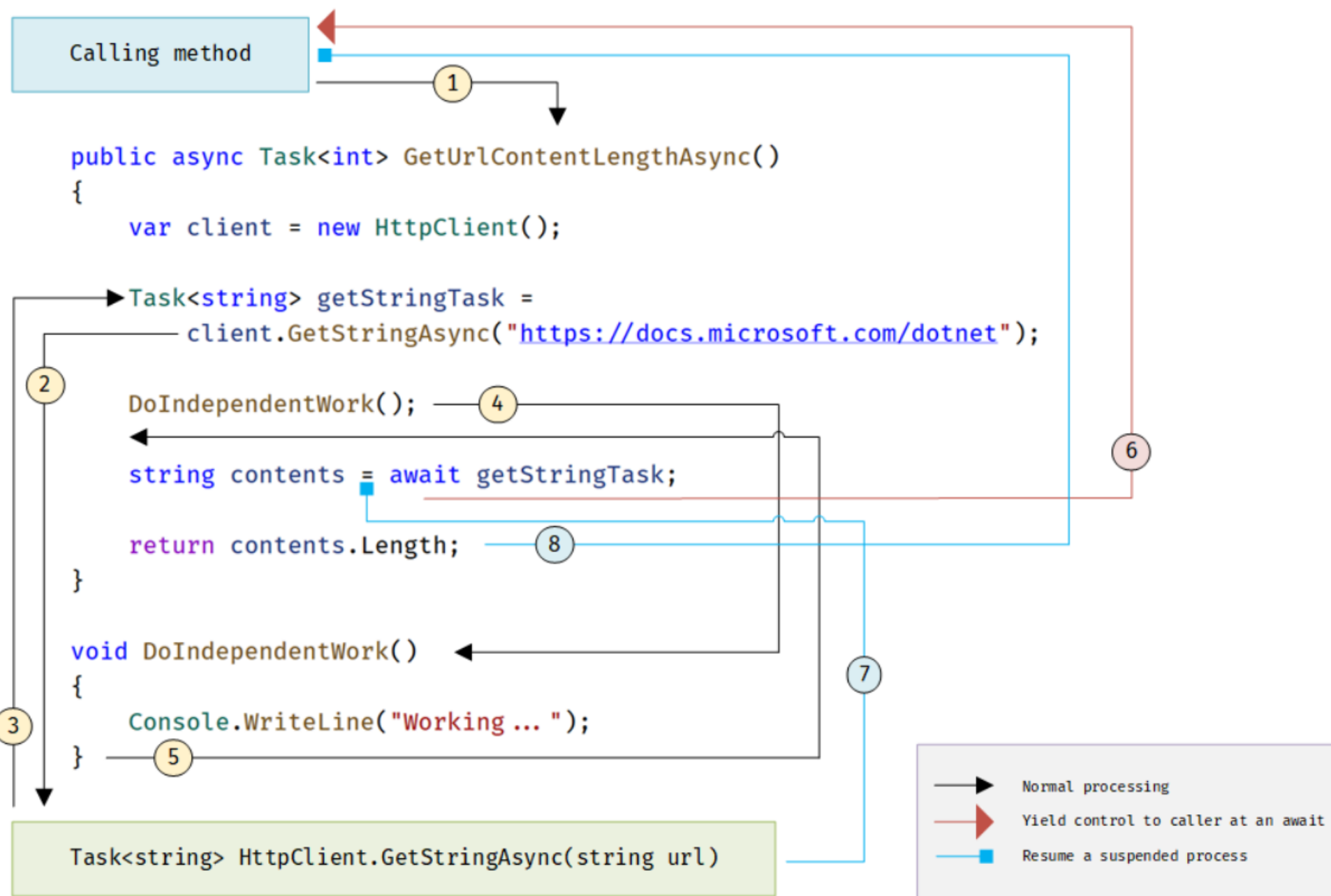


Когато задачи чакат и се изпълняват последователно



Само 1 нишка!

Какво се случва в един async метод



Стъпка 1.Извикващият метод извиква и изчаква асинхронния метод `GetUrlContentLengthAsync`.

Стъпка 2.Методът `GetUrlContentLengthAsync` създава обект `HttpClient` и извиква асинхронния метод `GetStringAsync`, който трябва да изтегли съдържанието на даден уебсайт като текст.

Стъпка 3.В `GetStringAsync` възниква операция, която изисква изчакване, например зареждане на информация от интернет. Вместо да блокира изпълнението, методът временно предава управлението обратно.

Стъпка 4.`GetStringAsync` връща обект от тип `Task<string>`. Този обект представя текуща асинхронна операция, която по-късно ще върне резултат от тип `string`. В `GetUrlContentLengthAsync` тази задача се записва в променливата `getStringTask`.

Стъпка 5.Понеже `getStringTask` все още не е изчакана чрез `await`, методът `GetUrlContentLengthAsync` може да продължи с друга работа, която не зависи от резултата. В примера това е извикването на метода `DoIndependentWork`.

Стъпка 6.`DoIndependentWork` е обикновен синхронен метод. Той изпълнява своята работа и приключва.

Стъпка 7.След това `GetUrlContentLengthAsync` стига до момент, в който вече му е нужен резултатът от `getStringTask`, за да продължи. Той иска да изчисли дължината на получения текст, но това е невъзможно, преди текстът да бъде получен.

Стъпка 8.Затова `GetUrlContentLengthAsync` използва `await getStringTask`. Така методът временно спира своето изпълнение и връща управление към извикващия метод, без да блокира нишката.

Стъпка 9. Когато асинхронната операция приключи и текстът бъде изтеглен, изпълнението на `GetUrlContentLengthAsync` продължава от мястото след `await`.

Стъпка 10. Накрая методът изчислява дължината на текста чрез `contents.Length` и връща резултат от тип `int`.

Най-важната идея:

При **`async`** и **`await`** методът не стои блокиран, докато чака резултат. Вместо това той временно прекъсва изпълнението си, връща управление на извикващия код и продължава по-късно, когато резултатът стане готов.